

# Laboratoire 3 : Git, GitHub et travail d'équipe

---

## Contexte

Ce laboratoire est la suite directe du laboratoire 2. Vous allez reprendre une application Node.js/React/Material UI créée avec un agent IA et l'améliorer en équipe en utilisant Git et GitHub.

L'objectif n'est pas de créer une nouvelle application, mais de mettre en place un flux de travail collaboratif réaliste autour d'une application existante.

## Objectifs

À la fin du laboratoire, vous devrez être capable de :

- utiliser Git pour suivre l'évolution d'une application;
- créer, nommer et fusionner des branches;
- synchroniser un dépôt local avec GitHub;
- créer et traiter une pull request;
- effectuer une revue de code;
- collaborer avec un autre développeur sans travailler directement dans la branche principale;
- utiliser des commits et des tags pour identifier les versions d'une application.

## Organisation

Le laboratoire se fait en équipe de deux.

Chaque équipe choisit une des applications réalisées au laboratoire 2 comme projet de référence. Le propriétaire du dépôt ajoute son coéquipier comme collaborateur sur GitHub. Le dépôt doit être accessible à l'enseignant; s'il est privé, ajoutez `nicolas.payre@cegepsherbrooke.qc.ca` comme collaborateur.

Les deux étudiants doivent réaliser une contribution identifiable. Une même personne ne doit pas faire toutes les étapes à la place de son coéquipier.

## Travail à faire

### Tâche 1 : Préparer le projet existant

1. Choisir une application du laboratoire 2 qui fonctionne suffisamment pour être modifiée.

2. Vérifier que le dépôt contient au minimum le code de l'application, un `README.md` et un `.gitignore` approprié.
3. Vérifier que l'application peut être installée et démarrée à partir d'un nouveau clonage du dépôt.
4. Créer un point de départ propre dans la branche `main` et ajouter le tag `lab2-final` pour identifier la version remise au laboratoire 2.
5. Ajouter le coéquipier comme collaborateur du dépôt GitHub.
6. Chaque étudiant doit cloner le dépôt ou récupérer la version à jour et vérifier la connexion avec le dépôt distant.

## Tâche 2 : Première contribution de chaque étudiant

Chaque étudiant doit réaliser une amélioration différente et pertinente de l'application. La modification doit être visible ou vérifiable; elle peut par exemple être :

- l'ajout d'une nouvelle conversion;
- l'amélioration de la validation des données;
- l'amélioration de l'interface ou de l'expérience utilisateur;
- l'ajout d'un composant réutilisable;
- l'ajout de tests ou d'une documentation technique;
- la correction d'un problème réel observé dans l'application.

Pour sa contribution, chaque étudiant doit :

1. Mettre à jour `main` avant de créer sa branche.
2. Créer une branche descriptive, par exemple `feature/ajout-conversion` ou `feature/validation-formulaire`.
3. Réaliser sa modification dans cette branche.
4. Tester l'application localement.
5. Créer au moins un commit dont le message décrit clairement la modification.
6. Pousser la branche vers GitHub.

## Tâche 3 : Pull request et revue de code

Pour chacune des deux contributions :

1. Créer une pull request de la branche de fonctionnalité vers `main`.
2. Décrire le problème traité, la solution proposée et les tests effectués.
3. Demander au coéquipier de faire la revue de code.

4. Le réviseur doit laisser au moins un commentaire pertinent : question, suggestion, vérification ou demande d'amélioration.
5. Répondre aux commentaires et ajouter un commit supplémentaire si une correction est nécessaire.
6. Vérifier que l'application fonctionne toujours après les changements.
7. Faire approuver puis fusionner la pull request dans `main`.
8. Supprimer la branche distante et la branche locale seulement après la fusion.

Si les deux contributions créent un conflit, l'équipe doit le résoudre et expliquer brièvement la résolution dans le document de remise.

#### Tâche 4 : Version finale

1. Récupérer la version fusionnée de `main`.
2. Tester l'application complète après l'intégration des deux contributions.
3. Mettre à jour le `README.md` avec une description de l'application, les instructions d'installation et les fonctionnalités ajoutées.
4. Créer le tag `lab3-final` sur la version finale.
5. Pousser la branche principale et le tag vers GitHub.

#### Règles importantes

- Les contributions doivent être faites dans des branches de fonctionnalité, jamais directement dans `main`.
- Les messages de commit doivent être explicites et correspondre réellement aux changements.
- Chaque étudiant doit avoir ses propres commits et sa propre pull request.
- Les modifications doivent porter sur l'application du laboratoire 2, et non sur un projet sans lien.
- Vous pouvez utiliser Codex ou un autre outil d'IA, mais vous devez être capables d'expliquer les changements réalisés et les commandes Git utilisées.
- Ne publiez jamais de clé, mot de passe, jeton ou autre secret dans le dépôt.

#### Remise

Remettez dans Léa un document Markdown ( `.md` ) contenant :

- le nom des deux coéquipiers;
- le lien vers le dépôt GitHub;

- le lien vers la pull request de chaque étudiant;
- le nom de la branche créée par chaque étudiant;
- le ou les messages de commit principaux de chaque contribution;
- le nom du tag final;
- une courte description de la contribution de chaque étudiant;
- les tests effectués après chaque contribution et après la fusion;
- une brève explication de tout conflit rencontré et de sa résolution, s'il y en a eu.

Une seule remise par équipe est nécessaire. Le dépôt doit contenir l'historique Git complet.

## Critères de réussite

- Le dépôt de l'application du laboratoire 2 est correctement utilisé.
- L'historique contient un point de départ identifiable et un tag `lab2-final`.
- Chaque étudiant a réalisé une contribution identifiable dans sa propre branche.
- Les deux contributions ont fait l'objet d'une pull request et d'une revue de code.
- Les changements ont été fusionnés dans `main` et le tag `lab3-final` est présent sur GitHub.
- L'application fonctionne après l'intégration.
- Le document de remise contient les liens et les preuves demandées.